Inverted Pendulum Control Game

Comprehensive Technical Documentation

Project: `cart_pole_control_game`

March 14, 2026

Contents

1 Project Purpose and Scope **4**

2 Why the Inverted Pendulum? **4**

3 System Modeling **4**
- 3.1 State, Input, and Sign Convention . . . 4
- 3.2 Physical Parameters . . . 5
- 3.3 Nonlinear Dynamics Used in Code . . . 5
- 3.4 Lagrangian Background (Conceptual) . . . 5

4 Numerical Simulation and Constraints **5**
- 4.1 Integration . . . 5
- 4.2 Applied Input and Saturation . . . 6
- 4.3 Track Boundary and Failure Condition . . . 6

5 Linearization About Upright **6**
- 5.1 Linear Model . . . 6
- 5.2 Interpretation . . . 6

6 PID Controller in This Project **6**
- 6.1 Classical PID . . . 6
- 6.2 Project-Specific Dual-Loop PID Law . . . 7
- 6.3 Anti-Windup and Initialization . . . 7
- 6.4 Algorithmic Nature . . . 7

7 LQR Controller in This Project **7**
- 7.1 Optimal Control Problem . . . 7
- 7.2 CARE and Feedback Law . . . 7
- 7.3 How CARE Is Solved Here . . . 8
- 7.4 Closed-Loop Eigenvalues (Display) . . . 8
- 7.5 Runtime Policy and Live Recompute . . . 8

8 MPC Controller in This Project **8**
- 8.1 Optimization Problem . . . 8
- 8.2 Horizon Definitions . . . 9
- 8.3 Solver Used Here . . . 9
- 8.4 Online Complexity . . . 9
- 8.5 MPC Output to UI . . . 9

9 Controller Data Path and "Instant Responsiveness" **9**
- 9.1 Runtime Pipeline . . . 10
- 9.2 Per-Controller Gain/Action Update Path . . . 10

10 Metrics, Logging, and Visualization **10**
10.1 Metrics 10
10.2 CSV Logging 10
10.3 Rendered Technical Panels 10

11 File-Level Architecture **11**

12 Known Implementation Notes **11**

13 Practical Comparison of Controllers in This Project **12**

1 Project Purpose and Scope

This project is an in-browser, zero-dependency control-theory simulator for an inverted pendulum on a cart (cart-pole). It compares three classical control strategies in real time:

1. PID (dual-loop parallel form),
2. LQR (linear-quadratic regulator on a linearized model),
3. MPC (finite-horizon nonlinear shooting with gradient descent).

The simulator integrates nonlinear dynamics with RK4, renders all telemetry on a canvas UI, allows live gain tuning, and logs per-tick data for download.

2 Why the Inverted Pendulum?

The inverted pendulum is a canonical benchmark for feedback control because:

1. It is open-loop unstable around the upright equilibrium.
2. It is nonlinear and coupled (cart position and pole angle are dynamically linked).
3. It has practical constraints (force limits, track limits), mirroring real systems.
4. It cleanly demonstrates differences between model-free feedback (PID), optimal linear control (LQR), and constrained predictive control (MPC).

3 System Modeling

3.1 State, Input, and Sign Convention

The simulator uses the state vector

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & \theta & \dot{\theta} \end{bmatrix}^\top, \tag{1}$$

where:

1. x: cart position,
2. $\dot{x}$: cart velocity,
3. θ: pole angle from *upright* ($\theta = 0$ is perfectly upright),
4. $\dot{\theta}$: angular velocity.

Control input:

$$u = \text{horizontal force applied to the cart (N)}. \tag{2}$$

In this implementation, a positive angle (pole leaning right) requires a positive corrective push (cart pushed right) to move under the pole.

3.2 Physical Parameters

Default parameters are loaded from `js/config.js`.

`Symbol`	Default	Meaning
M	1.0	cart mass (kg)
m	0.3	pole mass (kg)
L	0.6	pole half-length parameter (m)
g	9.81	gravity (m/s^2)
b	0.1	cart viscous friction coefficient
$U_{\max}$	30	actuator limit used by controllers (N)
$dt_{\texttt{inner}}$	0.002	physics sub-step (s), nominal 500 Hz

3.3 Nonlinear Dynamics Used in Code

Dynamics are implemented in `js/physics.js` as:

$$D(\theta) = M + m\sin^2\theta, \tag{3}$$

$$\ddot{x} = \frac{u - b\dot{x} + mL\dot{\theta}^2\sin\theta + mg\sin\theta\cos\theta}{D(\theta)}, \tag{4}$$

$$\ddot{\theta} = \frac{-(M+m)g\sin\theta - \cos\theta\left(u - b\dot{x} + mL\dot{\theta}^2\sin\theta\right)}{LD(\theta)}. \tag{5}$$

Thus the state derivative is

$$\dot{\mathbf{x}} = f(\mathbf{x}, u) = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix}. \tag{6}$$

3.4 Lagrangian Background (Conceptual)

These equations are the standard cart-pole Lagrange equations in manipulated form:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = Q_i, \tag{7}$$

with generalized coordinates $q = \{x, \theta\}$, $\mathcal{L} = T - V$, and generalized force on cart equal to $u - b\dot{x}$.

4 Numerical Simulation and Constraints

4.1 Integration

The continuous dynamics are integrated using fourth-order Runge–Kutta (RK4):

$$\mathbf{x}_{k+1} = \mathrm{RK4}(\mathbf{x}_k, u_k, \Delta t)\,. \tag{8}$$

At render frame time, the simulator computes a frame Δt and subdivides it into many physics sub-steps near $dt_{\text{inner}} = 0.002$ s.

4.2 Applied Input and Saturation

At each frame:

1. Controller computes u.
2. Manual push / challenge disturbance is added.
3. Final input is clamped in `main.js` to

$$u \in [-1.5U_{\max},\ 1.5U_{\max}]. \tag{9}$$

Inside MPC rollout and MPC cost evaluation, each candidate control is clamped to

$$u_k \in [-U_{\max}, U_{\max}]. \tag{10}$$

4.3 Track Boundary and Failure Condition

1. Soft track wall: when $|x|$ exceeds `trackLen`, velocity is damped and position clamped.
2. Episode fails when

$$|\theta| > 0.65\pi \approx 117^\circ. \tag{11}$$

5 Linearization About Upright

5.1 Linear Model

The project linearizes at the upright equilibrium:

$$\theta = 0, \quad \dot{x} = 0, \quad \dot{\theta} = 0, \quad u = 0. \tag{12}$$

Resulting continuous-time linear model:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u, \tag{13}$$

with

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -b/M & mg/M & 0 \\ 0 & 0 & 0 & 1 \\ 0 & b/(LM) & -(M+m)g/(LM) & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1/M \\ 0 \\ -1/(LM) \end{bmatrix}. \tag{14}$$

5.2 Interpretation

This linearization is accurate near upright and is used by LQR for analytic gain synthesis. MPC uses the *nonlinear* model for prediction.

6 PID Controller in This Project

6.1 Classical PID

For tracking error $e(t)$, ideal PID is

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau)d\tau + K_D \frac{de(t)}{dt}. \tag{15}$$

6.2 Project-Specific Dual-Loop PID Law

This project uses two coupled PID-like loops with state feedback:

$$u_\theta = K_{p\theta}\theta + K_{i\theta}\int \theta\, dt + K_{d\theta}\dot{\theta}, \tag{16}$$

$$u_x = -\left(K_{px}x + K_{ix}\int x\, dt + K_{dx}\dot{x}\right), \tag{17}$$

$$u = u_\theta + u_x. \tag{18}$$

Default gains:

$$\{K_{p\theta}, K_{i\theta}, K_{d\theta}, K_{px}, K_{ix}, K_{dx}, i_{\max}\} = \{64, 2, 9.8, 10, 0.5, 15.3, 8\}. \tag{19}$$

6.3 Anti-Windup and Initialization

Integral states are clamped each tick:

$$I_\theta \leftarrow \mathrm{clip}(I_\theta + \theta\, dt,\ -i_{\max},\ i_{\max}), \tag{20}$$

$$I_x \leftarrow \mathrm{clip}(I_x + x\, dt,\ -i_{\max},\ i_{\max}). \tag{21}$$

On first call after reset/mode switch, PID returns $u = 0$ once (`_pidNeedsInit`) to avoid transients from stale integral state.

6.4 Algorithmic Nature

PID in this code does *not* solve an optimization problem and does not use a numerical solver for gains online. It evaluates algebraic formulas each frame: computational cost $O(1)$.

7 LQR Controller in This Project

7.1 Optimal Control Problem

For linearized dynamics $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$, continuous-time infinite-horizon LQR minimizes:

$$J = \int_0^\infty \left(\mathbf{x}^\top \mathbf{Q}\mathbf{x} + u^\top \mathbf{R} u\right) dt, \tag{22}$$

with

$$\mathbf{Q} = \mathrm{diag}(Q_x, Q_{\dot{x}}, Q_\theta, Q_{\dot{\theta}}), \quad \mathbf{R} = [R]. \tag{23}$$

Defaults:

$$\mathbf{Q} = \mathrm{diag}(1, 0.8, 20, 2.5), \quad R = 0.01. \tag{24}$$

7.2 CARE and Feedback Law

LQR gain is obtained from continuous-time algebraic Riccati equation (CARE):

$$\mathbf{A}^\top P + P\mathbf{A} - P\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top P + \mathbf{Q} = 0, \tag{25}$$

then

$$K = \mathbf{R}^{-1}\mathbf{B}^\top P, \qquad u = -K\mathbf{x}. \tag{26}$$

7.3 How CARE Is Solved Here

This project uses a custom iterative integration of the Riccati differential equation toward steady state:

$$\dot{P} = \mathbf{A}^\top P + P\mathbf{A} - P\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top P + \mathbf{Q}. \tag{27}$$

Implementation details:

1. Initialize $P_0 = \mathrm{diag}(Q_x, Q_{\dot{x}}, Q_\theta, Q_{\dot{\theta}})$.
2. Euler update $P \leftarrow P + dt \cdot \dot{P}$ with $dt = 10^{-4}$.
3. Stop when $\|\dot{P}\|_F$ is sufficiently small (or max iterations reached).

This is a custom numeric solver in `js/lqr.js`; no external linear algebra library is used.

7.4 Closed-Loop Eigenvalues (Display)

For display only, the code computes eigenvalues of

$$\mathbf{A}_{\mathrm{cl}} = \mathbf{A} - \mathbf{B}K \tag{28}$$

by:

1. Building characteristic polynomial via Newton identities.
2. Solving degree-4 polynomial roots using Durand–Kerner.

This verifies stability ($\Re\{\lambda_i\} < 0$).

7.5 Runtime Policy and Live Recompute

1. Startup: solve LQR once, store `sim.lqrData`.
2. Runtime: each tick applies $u = -K\mathbf{x}$ in $O(1)$.
3. Tuner changes to Q/R: trigger debounced re-solve after 120 ms.

8 MPC Controller in This Project

8.1 Optimization Problem

This is receding-horizon nonlinear MPC with direct shooting over control sequence

$$U = \{u_0, \ldots, u_{N_h-1}\}. \tag{29}$$

Cost implemented in `mpcCost`:

$$J(U) = \sum_{k=0}^{N_h-1} \left(Q_x x_k^2 + Q_{\dot{x}} \dot{x}_k^2 + Q_\theta \theta_k^2 + Q_{\dot{\theta}} \dot{\theta}_k^2 + R u_k^2\right) + Q_f \left(Q_x x_N^2 + Q_{\dot{x}} \dot{x}_N^2 + Q_\theta \theta_N^2 + Q_{\dot{\theta}} \dot{\theta}_N^2\right), \tag{30}$$

with $Q_f = 15$.

Dynamics constraint in prediction:

$$\mathbf{x}_{k+1} = f_{\mathrm{RK4}}(\mathbf{x}_k, u_k, dt_{\mathrm{MPC}}), \quad u_k \in [-U_{\max}, U_{\max}]. \tag{31}$$

8.2 Horizon Definitions

1. Prediction horizon steps: $N_h = 22$ (default).
2. MPC time step: $dt_{\text{MPC}} = 0.025$ s.
3. Look-ahead duration: $N_h dt_{\text{MPC}} = 0.55$ s.
4. State horizon: $N_h + 1$ states (x_0 to x_{N_h}).
5. Control horizon: N_h (all horizon controls are optimized).

8.3 Solver Used Here

This MPC uses a **custom finite-difference batch gradient-descent shooting solver**:

1. Warm start:
   1.1. first call or reset: seed sequence by rolling out LQR policy,
   1.2. subsequent calls: shift old sequence left (receding horizon), copy tail.
2. For each iteration ($N_i = 12$ default):
   2.1. compute base cost J_0,
   2.2. for each index k, perturb u_k by $\epsilon = 1.0$ and estimate

$$\frac{\partial J}{\partial u_k} \approx \frac{J(U + \epsilon e_k) - J(U)}{\epsilon}, \tag{32}$$

   2.3. batch update all controls:

$$u_k \leftarrow \text{clip}(u_k - \eta g_k,\ -U_{\max}, U_{\max}), \quad \eta = 0.5. \tag{33}$$

3. Apply only u_0 to plant; keep $u_1, \ldots, u_{N_h-1}$ for next frame.

8.4 Online Complexity

Finite-difference MPC here is approximately $O(N_h^2 N_i)$ due repeated horizon rollouts per gradient component.

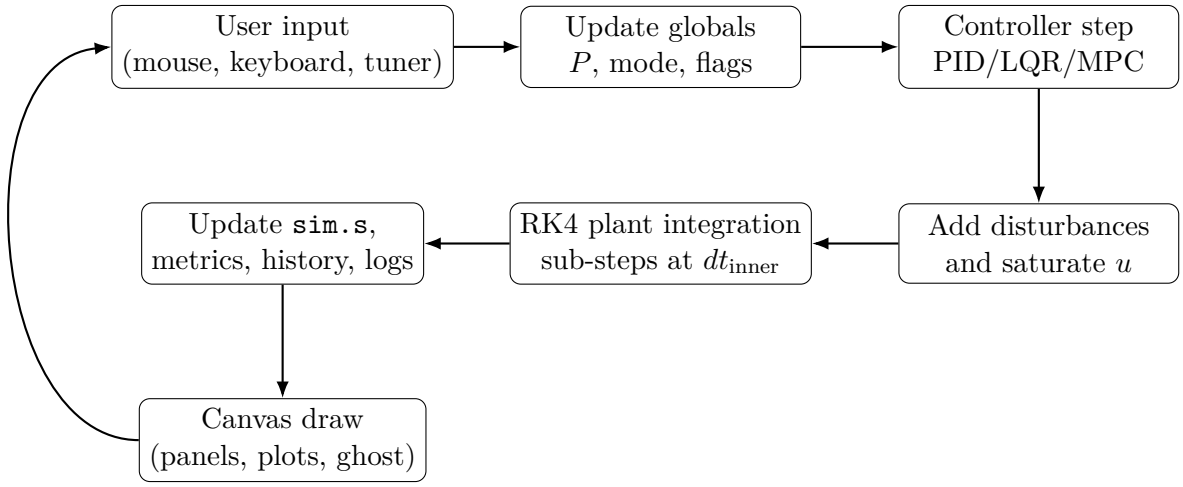
8.5 MPC Output to UI

Current optimized sequence is also rolled out in `mpcPredict` and drawn as a ghost trajectory, making the prediction visible in the canvas.

9 Controller Data Path and "Instant Responsiveness"

All controller code, physics, and rendering live in the same browser JS runtime (no backend). Responsiveness comes from shared in-memory state updated every animation frame.

9.1 Runtime Pipeline



9.2 Per-Controller Gain/Action Update Path

PID Tuner slider directly edits `P.pid.*`; next `tick()` uses new gains immediately.

LQR Tuner edits Q/R then debounced `solveLQR()` recomputes K and stores `sim.lqrData`; next tick applies new $u = -Kx$.

MPC Tuner edits $Q/R/N_h$ and sets `_mpcNeedsInit`; next `mpcStep()` reseeds/re-optimizes sequence and applies new u_0.

10 Metrics, Logging, and Visualization

10.1 Metrics

For each controller mode, the simulator tracks:

1. balance time,
2. settling time (angle below threshold for required duration),
3. peak $|\theta|$,
4. control effort $\int u^2 dt$ (numerical accumulation).

10.2 CSV Logging

Every simulation tick logs:

$$\texttt{run, t, x, xdot, theta_rad, thetadot, u, alive.} \tag{34}$$

10.3 Rendered Technical Panels

The UI displays:

1. current state vector and force,
2. linearized matrices $\mathbf{A}, \mathbf{B}$,

3. closed-loop eigenvalues for LQR,
4. angle history plot,
5. control history plot,
6. phase portrait $(\theta, \dot{\theta})$,
7. controller comparison table,
8. MPC predicted ghost trajectory.

11 File-Level Architecture

File	Responsibility
`index.html`	Entry point, script load order, loading overlay, tuner init.
`js/config.js`	Global parameter object `P`.
`js/physics.js`	Nonlinear dynamics, RK4 integrator, linearization matrices.
`js/lqr.js`	CARE iterative solver, K synthesis, eigenvalue utilities.
`js/mpc.js`	Receding-horizon MPC with finite-difference gradient descent.
`js/pid.js`	Dual-loop PID and anti-windup integrator state.
`js/main.js`	Simulation state, tick/frame loop, integration, metrics, logs.
`js/render.js`	Full canvas rendering for desktop and mobile layouts.
`js/interaction.js`	Keyboard/mouse/touch handling and mode switching.
`js/tuner.js`	Floating live tuner UI and live parameter update logic.
`css/style.css`	Global page/canvas styling and loading overlay style.
`css/tuner.css`	Tuner panel styling and responsive behavior.

12 Known Implementation Notes

1. The runtime app is the modular stack loaded by `index.html`; the file `cartpole` is a standalone legacy snapshot and is not the active entrypoint.
2. Keyboard binding for PID switch in code is `KeyI`; if documentation says `P` for PID, that is inconsistent with current implementation.
3. `mpcU` is allocated once using initial `P.Nh`; increasing `Nh` above startup value without reallocating can lead to invalid MPC values.

13 Practical Comparison of Controllers in This Project

Controller	Mathematical basis	Online computation	Strengths/limits here
PID	Heuristic feedback + integral memory	$O(1)$	Simple and robust; no model-based optimality or explicit constraints.
LQR	Linear optimal control from CARE	$O(1)$	Very efficient; optimal for linearized model, less accurate far from upright.
MPC	Finite-horizon nonlinear optimization	Heavy ($O(N_h^2 N_i)$ approx.)	Handles nonlinear rollout and constraints directly; higher compute cost.

Conclusion

This project is a compact but technically rich control-systems lab. It combines:

1. nonlinear plant simulation,
2. linear model-based optimal control (LQR),
3. nonlinear constrained predictive optimization (MPC),
4. practical feedback design (PID),
5. live interactive visualization and tuning.

As implemented, it is ideal for studying controller trade-offs between simplicity, optimality assumptions, and real-time computational cost.